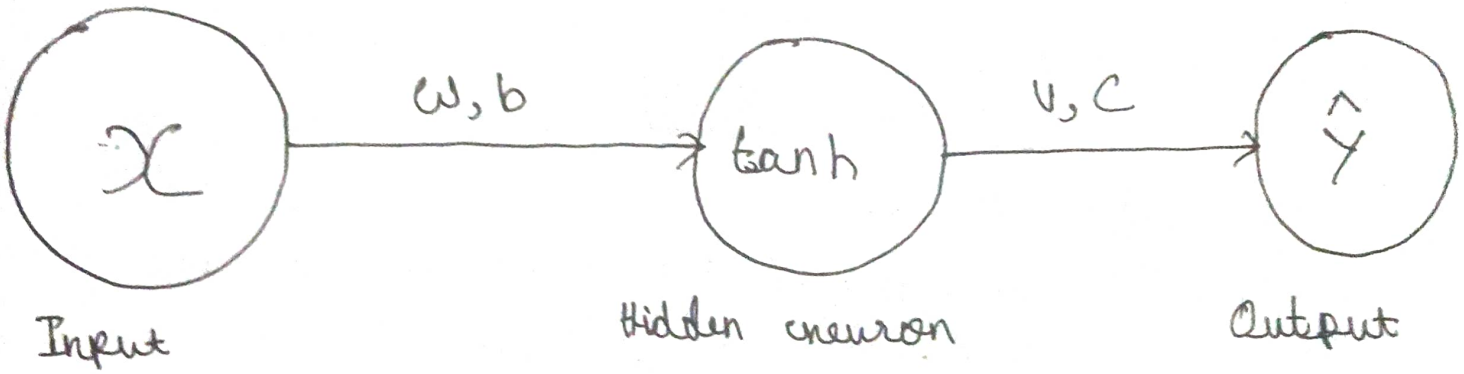
Derivation of Backpropagation in a Single Hidden-Neuron Neural Network

Objective

The objective is to implement a simple neural network and understand how backpropagation calculates gradients to update the weights and biases using gradient descent.

Network Architecture



Consider a neural network containing

- One input neuron: x
- One hidden neuron with a tanh activation function
- One output neuron with a linear activation function

Initial Parameters

Given: $x = 1$, $y = 2$, $\eta = 0.1$

Initial weight and biases:

$$w = 0.5, \quad b = 0, \quad v = 1, \quad c = 0$$

Here, y is the target output and η is the learning rate

Forward Propagation

Step - 1 Calculate the hidden neuron input

$$z = wx + b$$
$$= (0.5)(1) + 0 = 0.5$$

Step - 2 Apply the tanh activation function

$$h = \tanh(z)$$
$$= \tanh(0.5) = 0.462117$$

Step - 3 Calculate the predicted output

$$\hat{y} = vh + c$$
$$= (1)(0.462117) + 0 = 0.462117$$

$$\boxed{\hat{y} = 0.462117}$$

Step - 4 Calculate the loss

Using mean squared error for one sample

$$J = \frac{1}{2}(\hat{y} - y)^2 \Rightarrow \frac{1}{2}(0.462117 - 2)^2$$
$$= 1.18254$$

Backpropagation and Gradient Calculation

Let the output error be

$$\delta = \hat{y} - y \Rightarrow 0.462117 - 2$$
$$= -1.537882$$

The derivative of tanh activation function is

$$\frac{dh}{dz} = 1 - h^2 \Rightarrow 1 - (0.462117)^2$$
$$= 0.786447$$

The required gradients are derived using the chain rule

$$\frac{dJ}{dc} = \delta \quad \frac{dJ}{dv} = \delta h \quad \left| \frac{dJ}{dw} = \delta . v . (1 - h^2) . x \right.$$
$$\left. \frac{dJ}{db} = \delta . v . (1 - h^2) \right.$$

Calculate the Gradients

Gradient with respect to output bias C

$$\frac{\partial J}{\partial C} = -1.537882$$

Gradient with respect to output weight V

$$\frac{\partial J}{\partial V} = (-1.537882)(0.469117)$$

$$= -0.710682$$

Gradient with respect to hidden weight W

$$\frac{\partial J}{\partial W} = (-1.537882)(1)(0.786447)(1)$$

$$= -1.209464$$

Gradient with hidden bias b

$$\frac{\partial J}{\partial b} = (-1.537882)(1)(0.786447)$$

$$= -1.2094644$$

Gradient Decent Weight Update

The gradient decend update rule is

$$\theta_{new} = \theta_{old} - \alpha \frac{\partial J}{\partial \theta} \quad \text{where } \alpha = 0.1$$

Update hidden weight W

$W_{new} = 0.5 - 0.1(-1.20946) \Rightarrow 0.620946$

Update hidden bias b

$b_{new} = 0 - 0.1(-1.20946) \Rightarrow 0.120946$

Update output weight V

$V_{new} = 1 - 0.1(-0.710682) \Rightarrow 1.0710682$

Update output bias C

$C_{new} = 0 - 0.1(-1.537882) \Rightarrow 0.1537882$

Calculate the New Output and loss

Using the updated parameters

$$Z_{new} = W_{new} x + b_{new} = 0.692094(1) + 0.130946$$
$$= 0.741892$$

Apply tanh $\Rightarrow$ $h_{new} = \tanh(0.741892)$

$$h_{new} \approx 0.630287442 2$$

Calculate new predicted output

$$\hat{Y}_{new} = V_{new} h_{new} + C_{new}$$

$$\boxed{\hat{Y}_{new} \approx 0.828869}$$

Calculate the new loss:

$$J_{new} = \frac{1}{2}(\hat{Y}_{new} - Y)^2$$

$$\boxed{J_{new} \approx 0.685737}$$

Conclusion

Backpropagation calculates the gradients of the loss function with respect to each weight and bias using the chain rule. Gradient descent then updates these parameters in the opposite direction of the gradients

In this example the loss decreased from approx 1.18254 to 0.68577 after one update. This demonstrates how backpropagation helps the neural network reduce prediction error during training